

Clase 107 — Server-Side Template Injection (SSTI)

Parte: 4 — Seguridad de aplicaciones web · Fuente: Real-World Bug Hunting (Yaworski) / PortSwigger Research 🕒 Duración estimada: 110 min · Nivel: Experto

🎯 Objetivo

Explotar la **inyección de plantillas del lado servidor (SSTI)**: cuando el input del usuario se evalúa como parte de una plantilla (Jinja2, Twig, Freemarker, etc.), permitiendo desde filtración de datos hasta RCE. Es un fallo potente que aparece en emails personalizados, generación de documentos y paneles configurables.

⚠️ **Ética:** puede derivar en RCE. Solo en labs propios/autorizados (PortSwigger). Nunca en sistemas ajenos.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Detectar** SSTI con payloads de prueba matemáticos.
2. **Identificar** el motor de plantillas por su comportamiento.
3. **Escalar** de evaluación de expresiones a lectura de archivos y RCE.
4. **Construir** payloads específicos por motor (Jinja2, Twig, Freemarker).
5. **Recomendar** sandboxing y separación de datos y plantilla.

📖 Temas

#	Tema	Por qué importa
1	Motores de plantillas	Contexto del fallo
2	Detección (<code>{{7*7}}</code>)	Confirmar la inyección
3	Fingerprinting del motor	El exploit depende del motor
4	De expresión a RCE	Escalada de impacto
5	Jinja2 / Twig / Freemarker	Payloads concretos
6	SSTI vs. XSS	No confundir el sink
7	Defensa: sandbox, logic-less	Cierre del fallo

📖 Definiciones y características

- **SSTI:** el input se interpreta dentro de una plantilla del servidor. Característica: puede ejecutar código del lado servidor.

- **Motor de plantillas:** sistema que renderiza plantillas con datos (Jinja2, Twig...). Característica: cada uno tiene su sintaxis y capacidades.
- **Payload de detección:** expresión simple (`{{7*7}}`) que revela evaluación. Característica: si devuelve `49` , hay SSTI.
- **Fingerprinting:** identificar el motor por respuestas a payloads. Característica: guía el exploit específico.
- **Sandbox:** entorno restringido de ejecución de plantillas. Característica: mitiga (aunque a veces se evade) la RCE.
- **Logic-less templates:** motores sin ejecución de código (Mustache). Característica: reducen el riesgo por diseño.

Herramientas y preparación

- **PortSwigger labs** de SSTI.
- **tplmap** (automatización, con criterio).
- **Burp** para probar payloads.

```
git clone https://github.com/epinna/tplmap && cd tplmap && pip install -r requirements.txt
```

Laboratorio guiado

 Solo en labs propios.

1. Introduce `{{7*7}}` y `${7*7}` en campos que se reflejan (nombre, plantilla de email).
2. Si obtienes `49` , confirma SSTI e identifica el motor con payloads diferenciadores.
3. En Jinja2 (Python), escala a lectura de configuración y luego a ejecución:

```
{{ config.items() }}
{{ '.__class__.__mro__[1].__subclasses__()' }}
```

4. Encuentra un subclass que permita ejecutar comandos y ejecútalo en el lab.
5. En Twig/Freemarker, usa los payloads específicos del motor para leer archivos o ejecutar.
6. Confirma la RCE con una interacción OOB.
7. Documenta el motor, la cadena de escalada y el impacto.

Ejercicios

1. Diferencia la respuesta de `{{7*7}}` en SSTI de un simple reflejo de texto.
2. Haz fingerprinting distinguiendo Jinja2 de Twig con payloads.
3. Explica la cadena `__class__` → `__mro__` → `__subclasses__` en Python.
4. Diferencia SSTI de XSS: mismo `{{7*7}}` , distinto sink.
5. Usa tplmap y luego reproduce manualmente su payload.
6. Propón una arquitectura con plantillas logic-less para el caso de uso.

Reto verificable

Resuelve un lab de SSTI de PortSwigger que exija **fingerprint + RCE** y ejecuta un comando (p. ej. leer un archivo del sistema). **Criterio de aceptación:** el lab queda resuelto, documentas el motor identificado, la cadena de payloads y la defensa (sandbox, separar datos de plantilla, motor logic-less).

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
{{7*7}} sale literal	No hay SSTI ahí; puede ser XSS o texto plano
49 pero sin RCE	Motor con sandbox; busca bypass conocido
Payload de otro motor falla	Fingerprint incorrecto; ajusta el motor
tplmap no detecta	Contexto peculiar; prueba manual
RCE ciega	Usa OOB para confirmar

Preguntas frecuentes

 **¿SSTI siempre es RCE?** No siempre; depende del motor y su sandbox. A veces solo permite leer datos, pero muchos motores llegan a RCE.

 **¿Cómo distingo SSTI de XSS?** {{7*7}} que devuelve 49 indica evaluación en servidor (SSTI); si aparece literal y ejecuta JS, es XSS en cliente.

 **¿El sandbox me protege?** Ayuda, pero muchos sandboxes de plantillas han sido evadidos. La defensa robusta es no meter input del usuario en la plantilla.

Referencias

- Yaworski, *Real-World Bug Hunting*, cap. de template injection.
- PortSwigger SSTI research y labs: <https://portswigger.net/web-security/server-side-template-injection>
- OWASP Testing for SSTI (WSTG).

Siguiendo clase

Clase 108 - Vulnerabilidades en carga de archivos